

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №4
по дисциплине
«Управление данными»

Выполнил студент гр. ИВТб-2302-06-00

_____/Терешенко Н.А./

Проверил преподаватель

_____/Клюкин В.Л./

Киров

2026

Цель работы

Изучить способы связывания приложения на языке C# с базой данных под управлением СУБД PostgreSQL. Освоить практические навыки выполнения SQL-запросов, работы с библиотекой доступа к данным и разработки графического интерфейса.

Ход работы

Модуль доступа к базе данных на C#

В классе `DatabaseRepository` реализованы те же операции:

- `GetParentList` — формирует записи для раскрывающихся списков, склеивая ФИО и номер ID-карты;
- `FilterRecords` — выполняет регистронезависимый поиск по имени, фамилии или номеру ID-карты;
- `CheckExists` — проверяет существование пользователя с заданным номером ID-карты;
- `SaveRecord` — вызывает PL/pgSQL-функцию `save_users`, выполняющую вставку или обновление записи (UPSERT), и возвращает идентификатор пользователя;
- `DeleteRecord` — удаляет тариф через функцию `delete_tiers` с обработкой ошибок внешних ключей.

Для вызова этих методов из нативного кода предназначен класс `NativeExports`: методы помечаются атрибутом `[UnmanagedCallersOnly]` и экспортируются из сборки, компилируемой в нативную динамическую библиотеку. В файле проекта включена Ahead-of-Time компиляция, в результате чего получается файл `actions.so`.

Класс `DatabaseRepository`

```
using System;
using System.Collections.Generic;
using Npgsql;

public class DatabaseRepository
{
```

```

private readonly string _connectionString;

public DatabaseRepository(string connectionString)
{
    _connectionString = connectionString;
}

public List<KeyValuePair<long, string>> GetParentList()
{
    var result = new List<KeyValuePair<long, string>>();

    using (var conn = new NpgsqlConnection(_connectionString))
    {
        conn.Open();

        string sql = "SELECT user_id, first_name || ' ' || last_name || ' (' || i

        using (var cmd = new NpgsqlCommand(sql, conn))
        using (var reader = cmd.ExecuteReader())
        {
            while (reader.Read())
            {
                result.Add(new KeyValuePair<long, string>(
                    reader.GetInt64(0),
                    reader.GetString(1)
                ));
            }
        }
    }

    return result;
}

public List<string[]> FilterRecords(string searchText)
{

```

```

var records = new List<string[]>();

using (var conn = new NpgsqlConnection(_connectionString))
{
    conn.Open();

    string sql = @"SELECT user_id, first_name, last_name, phone_number, id_card_num
                    FROM users
                    WHERE LOWER(first_name) LIKE LOWER(@search)
                        OR LOWER(last_name) LIKE LOWER(@search)
                        OR LOWER(id_card_num) LIKE LOWER(@search)";

    using (var cmd = new NpgsqlCommand(sql, conn))
    {
        cmd.Parameters.AddWithValue("search", $"{searchText}%");

        using (var reader = cmd.ExecuteReader())
        {
            while (reader.Read())
            {
                records.Add(new string[]
                {
                    reader.GetInt64(0).ToString(),
                    reader.GetString(1),
                    reader.GetString(2),
                    reader.GetString(3),
                    reader.GetString(4)
                });
            }
        }
    }
}

return records;
}

```

```

public bool CheckExists(string idCardNum)
{
    using (var conn = new NpgsqlConnection(_connectionString))
    {
        conn.Open();
        string sql = "SELECT COUNT(1) FROM users WHERE id_card_num = @idc";

        using (var cmd = new NpgsqlCommand(sql, conn))
        {
            cmd.Parameters.AddWithValue("idc", idCardNum);
            long count = Convert.ToInt64(cmd.ExecuteScalar());
            return count > 0;
        }
    }
}

public long SaveRecord(string firstName, string lastName, string phoneNumber, string idCardNum)
{
    using (var conn = new NpgsqlConnection(_connectionString))
    {
        conn.Open();
        using (var cmd = new NpgsqlCommand("SELECT save_users(@fn, @ln, @pn, @idc", conn))
        {
            cmd.Parameters.AddWithValue("fn", firstName);
            cmd.Parameters.AddWithValue("ln", lastName);
            cmd.Parameters.AddWithValue("pn", phoneNumber);
            cmd.Parameters.AddWithValue("idc", idCardNum);

            return Convert.ToInt64(cmd.ExecuteScalar());
        }
    }
}

```

```

public bool DeleteRecord(int tierId, out string errorMessage)
{
    errorMessage = string.Empty;

    try
    {
        using (var conn = new NpgsqlConnection(_connectionString))
        {
            conn.Open();
            using (var cmd = new NpgsqlCommand("SELECT delete_tiers(@id)", conn))
            {
                cmd.Parameters.AddWithValue("id", tierId);
                cmd.ExecuteNonQuery();
                return true;
            }
        }
    }
    catch (PostgresException ex)
    {
        errorMessage = ex.MessageText;
        return false;
    }
}
}

```

Нативный экспорт функций

```

using System;
using System.Runtime.InteropServices;
using Npgsql;

public static class NativeExports
{
    private const string ConnString =
        "Host=127.0.0.1;Port=5432;Database=lab_db;Username=postgres;Password=postgres";
}

```

```

private static readonly DatabaseRepository Repo = new(ConnString);

[UnmanagedCallersOnly(EntryPoint = "save_user_native")]
public static long SaveUserNative(IntPtr fn, IntPtr ln, IntPtr pn, IntPtr idc)
{
    try
    {
        return Repo.SaveRecord(
            Marshal.PtrToStringUTF8(fn) ?? "",
            Marshal.PtrToStringUTF8(ln) ?? "",
            Marshal.PtrToStringUTF8(pn) ?? "",
            Marshal.PtrToStringUTF8(idc) ?? ""
        );
    }
    catch
    {
        return 0;
    }
}

```

```

[UnmanagedCallersOnly(EntryPoint = "check_exists_native")]
public static int CheckExistsNative(IntPtr idc)
{
    try
    {
        return Repo.CheckExists(Marshal.PtrToStringUTF8(idc) ?? "") ? 1 : 0;
    }
    catch
    {
        return 0;
    }
}

```

```

[UnmanagedCallersOnly(EntryPoint = "delete_tier_native")]
public static int DeleteTierNative(int tierId)
{
    try
    {
        return Repo.DeleteRecord(tierId, out _) ? 1 : 0;
    }
    catch
    {
        return 0;
    }
}

[UnmanagedCallersOnly(EntryPoint = "delete_user_native")]
public static int DeleteUserNative(long userId)
{
    try
    {
        using var conn = new NpgsqlConnection(ConnString);
        conn.Open();
        using var cmd = new NpgsqlCommand("DELETE FROM users WHERE user_id = @id", conn);
        cmd.Parameters.AddWithValue("id", userId);
        cmd.ExecuteNonQuery();
        return 1;
    }
    catch
    {
        return 0;
    }
}
}

```

Файл проекта

```
<Project Sdk="Microsoft.NET.Sdk">
```

```

<PropertyGroup>
  <OutputType>Library</OutputType>
  <TargetFramework>net10.0</TargetFramework>
  <ImplicitUsings>enable</ImplicitUsings>
  <Nullable>enable</Nullable>
  <AssemblyName>actions</AssemblyName>
</PropertyGroup>

<PropertyGroup>
  <PublishAot>true</PublishAot>
  <NativeLib>Shared</NativeLib>
</PropertyGroup>

<ItemGroup>
  <PackageReference Include="Npgsql" Version="10.0.3" />
</ItemGroup>

</Project>

```

Связка Python и C#

Исходный код

```

import ctypes
import os
import psycopg2
from psycopg2.extras import RealDictCursor

BACKEND_MODE = "python"
CONN_STRING = "dbname=lab_db user=postgres password=postgres host=127.0.0.1 port=5432"

CSHARP_LIB_PATH = os.path.abspath(
    "csharplab/bin/Release/net10.0/linux-x64/publish/actions.so"
)

```

```

csharp_lib = None

if os.path.exists(CSHARP_LIB_PATH):
    try:
        csharp_lib = ctypes.CDLL(CSHARP_LIB_PATH)

        csharp_lib.save_user_native.argtypes = [ctypes.c_char_p, ctypes.c_char_p, ctypes.c_int64]
        csharp_lib.save_user_native.restype = ctypes.c_int64

        csharp_lib.check_exists_native.argtypes = [ctypes.c_char_p]
        csharp_lib.check_exists_native.restype = ctypes.c_int

        csharp_lib.delete_tier_native.argtypes = [ctypes.c_int32]
        csharp_lib.delete_tier_native.restype = ctypes.c_int

        csharp_lib.delete_user_native.argtypes = [ctypes.c_int64]
        csharp_lib.delete_user_native.restype = ctypes.c_int
    except Exception as e:
        print(f"Warning: Could not load C# binary library: {e}")

def set_backend_mode(mode: str):
    global BACKEND_MODE
    if mode.lower() in ["python", "csharp"]:
        if mode.lower() == "csharp" and not csharp_lib:
            raise RuntimeError("C# shared object library is not loaded or missing.")
        BACKEND_MODE = mode.lower()
        print(f"Active Backend Switched To: {BACKEND_MODE.upper()}")
    else:
        raise ValueError("Invalid mode. Choose 'python' or 'csharp'.")

def _get_connection():
    return psycopg2.connect(CONN_STRING)

```

```

def save_user(first_name: str, last_name: str, phone: str, id_card: str) -> int:
    if BACKEND_MODE == "csharp":
        return csharp_lib.save_user_native(
            first_name.encode("utf-8"),
            last_name.encode("utf-8"),
            phone.encode("utf-8"),
            id_card.encode("utf-8")
        )

    sql = "SELECT save_users(%s, %s, %s, %s);"
    with _get_connection() as conn:
        with conn.cursor() as cursor:
            cursor.execute(sql, (first_name, last_name, phone, id_card))
            result = cursor.fetchone()
            conn.commit()
            return result[0] if result else None

def check_exists(id_card: str) -> bool:
    if BACKEND_MODE == "csharp":
        return bool(csharp_lib.check_exists_native(id_card.encode("utf-8")))

    sql = "SELECT COUNT(1) FROM users WHERE id_card_num = %s;"
    with _get_connection() as conn:
        with conn.cursor() as cursor:
            cursor.execute(sql, (id_card,))
            count = cursor.fetchone()[0]
            return count > 0

def delete_tier(tier_id: int) -> tuple[bool, str]:
    if BACKEND_MODE == "csharp":

```

```

        status = csharp_lib.delete_tier_native(tier_id)
        return (True, "") if status == 1 else (False, "Error executing C# delete oper

sql = "SELECT delete_tiers(%s);"
try:
    with _get_connection() as conn:
        with conn.cursor() as cursor:
            cursor.execute(sql, (tier_id,))
            conn.commit()
            return True, ""
except psycopg2.Error as ex:
    return False, ex.pgerror or str(ex)

def delete_user(user_id: int) -> tuple[bool, str]:
    if BACKEND_MODE == "csharp":
        status = csharp_lib.delete_user_native(user_id)
        return (True, "") if status == 1 else (False, "Error executing C# delete oper

sql = "DELETE FROM users WHERE user_id = %s;"
try:
    with _get_connection() as conn:
        with conn.cursor() as cursor:
            cursor.execute(sql, (user_id,))
            conn.commit()
            return True, ""
except psycopg2.Error as ex:
    return False, ex.pgerror or str(ex)

def filter_users(search_text: str) -> list[tuple]:
    sql = """
        SELECT user_id, first_name, last_name, phone_number, id_card_num
        FROM users

```

```

        WHERE LOWER(first_name) LIKE LOWER(%s)

        OR LOWER(last_name) LIKE LOWER(%s)

        OR LOWER(id_card_num) LIKE LOWER(%s);
"""

search_pattern = f"%{search_text}%"

with _get_connection() as conn:
    with conn.cursor() as cursor:
        cursor.execute(sql, (search_pattern, search_pattern, search_pattern))
    return cursor.fetchall()


def get_parent_list() -> list[tuple[int, str]]:
    sql = "SELECT user_id, first_name || ' ' || last_name || ' (' || id_card_num || '"
    with _get_connection() as conn:
        with conn.cursor() as cursor:
            cursor.execute(sql)
        return cursor.fetchall()

```

Графический интерфейс пользователя

Исходный код

```

import sys

import dual_manager

from PyQt6.QtWidgets import (
    QAbstractItemView,
    QApplication,
    QFormLayout,
    QHBoxLayout,
    QHeaderView,
    QLineEdit,
    QMainWindow,
    QMessageBox,
    QPushButton,
    QTableWidget,

```

```

    QTableWidgetItem,
    QVBoxLayout,
    QWidget,
)

```

```

class DatabaseManager:

```

```

    """Delegates all CRUD operations to the unified dual backend (python/C#)."""

```

```

    def __init__(self):

```

```

        self.backend = dual_manager

```

```

    def fetch_parent_records(self) -> list[tuple[int, str]]:

```

```

        return self.backend.get_parent_list()

```

```

    def filter_records(

```

```

        self, text_filter: str, start_date: str, end_date: str

```

```

) -> list[tuple]:

```

```

    del start_date, end_date

```

```

    return self.backend.filter_users(text_filter)

```

```

    def save_record(

```

```

        self, first_name: str, last_name: str, phone: str, id_card: str

```

```

) -> bool:

```

```

    if self.backend.check_exists(id_card):

```

```

        return False

```

```

    result = self.backend.save_user(first_name, last_name, phone, id_card)

```

```

    return result is not None and result > 0

```

```

    def delete_record(self, record_id: int) -> tuple[bool, str]:

```

```

        return self.backend.delete_user(record_id)

```

```

class MainWindow(QMainWindow):

```

```

def __init__(self):
    super().__init__()
    self.db = DatabaseManager()
    self.init_ui()

def init_ui(self):
    self.setWindowTitle("Управление данными БД")
    self.resize(900, 600)

    central_widget = QWidget()
    self.setCentralWidget(central_widget)
    main_layout = QVBoxLayout(central_widget)

    # -----
    # 1. Filter Section (Выборка и фильтрация)
    # -----
    filter_layout = QHBoxLayout()

    self.filter_input = QLineEdit()
    self.filter_input.setPlaceholderText("Поиск по имени, фамилии или ID-карте...")

    btn_apply_filter = QPushButton("Применить фильтр")
    btn_apply_filter.clicked.connect(self.on_filter)

    filter_layout.addWidget(self.filter_input)
    filter_layout.addWidget(btn_apply_filter)

    main_layout.addLayout(filter_layout)

    # -----
    # 2. Table Section (Отображение данных)
    # -----
    self.table = QTableWidgetItem()

```

```

self.table.setColumnCount(5)
self.table.setHorizontalHeaderLabels(
    ["ID", "Фамилия", "Имя", "Телефон", "ID-карта"]
)
self.table.horizontalHeader().setSectionResizeMode(
    QHeaderView.ResizeMode.Stretch
)
self.table.setSelectionBehavior(
    QAbstractItemView.SelectionBehavior.SelectRows
)
self.table.setEditTriggers(
    QAbstractItemView.EditTrigger.NoEditTriggers
)

main_layout.addWidget(self.table)

# -----
# 3. Input Section (Добавление/Редактирование)
# -----

form_layout = QFormLayout()

self.input_first_name = QLineEdit()
self.input_last_name = QLineEdit()
self.input_phone = QLineEdit()
self.input_id_card = QLineEdit()

form_layout.addRow("Имя:", self.input_first_name)
form_layout.addRow("Фамилия:", self.input_last_name)
form_layout.addRow("Телефон:", self.input_phone)
form_layout.addRow("ID-карта:", self.input_id_card)

main_layout.addLayout(form_layout)

# -----

```

```

# 4. Action Buttons (Управление)
# -----

actions_layout = QHBoxLayout()

btn_save = QPushButton("Сохранить запись")
btn_save.clicked.connect(self.on_save)

btn_delete = QPushButton("Удалить выбранное")
btn_delete.clicked.connect(self.on_delete)

actions_layout.addWidget(btn_save)
actions_layout.addWidget(btn_delete)

main_layout.addLayout(actions_layout)

# -----
# UI Logic & Event Handlers
# -----

def on_filter(self):
    """Handle search filtering execution via database backend."""
    query_text = self.filter_input.text().strip()

    results = self.db.filter_records(query_text, "", "")
    self.render_table_data(results)

    if not results:
        QMessageBox.information(self, "Поиск", "Такой записи нет")

def on_save(self):
    """Validate input and trigger PL/pgSQL save operation."""
    first_name = self.input_first_name.text().strip()
    last_name = self.input_last_name.text().strip()
    phone = self.input_phone.text().strip()
    id_card = self.input_id_card.text().strip()

```

```

if not first_name or not last_name or not phone or not id_card:
    QMessageBox.warning(
        self, "Ошибка ввода", "Все поля должны быть заполнены."
    )
    return

# Execute backend logic (checks duplicates before insert)
success = self.db.save_record(first_name, last_name, phone, id_card)

if success:
    QMessageBox.information(
        self, "Успех", "Запись успешно сохранена."
    )
    self.on_filter() # Refresh table
else:
    QMessageBox.critical(
        self,
        "Ошибка",
        "Запись с таким ID-карты уже существует или произошла ошибка записи."
    )

def on_delete(self):
    """Prompt confirmation and execute backend delete operation."""
    selected_rows = self.table.selectionModel().selectedRows()

    if not selected_rows:
        QMessageBox.warning(
            self,
            "Предупреждение",
            "Выберите строку для удаления из таблицы.",
        )
    return

```

```

row_index = selected_rows[0].row()
record_id = int(self.table.item(row_index, 0).text())

# Deletion confirmation prompt requirement
reply = QMessageBox.question(
    self,
    "Подтверждение удаления",
    "Вы действительно уверены?",
    QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No,
    QMessageBox.StandardButton.No,
)

if reply == QMessageBox.StandardButton.Yes:
    success, error = self.db.delete_record(record_id)
    if success:
        QMessageBox.information(
            self, "Успех", "Запись успешно удалена."
        )
        self.on_filter() # Refresh table
    else:
        QMessageBox.critical(
            self, "Ошибка", f"Не удалось удалить запись: {error or 'неизвестно'}"
        )

def render_table_data(self, records: list[tuple]):
    """Render records array into QTableWidgetItem rows."""
    self.table.setRowCount(0)
    for row_idx, row_data in enumerate(records):
        self.table.insertRow(row_idx)
        for col_idx, value in enumerate(row_data):
            self.table.setItem(
                row_idx, col_idx, QTableWidgetItem(str(value))
            )

```

```

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())

```

Результаты работы

На рис. 1 приведён полный список записей из базы данных, выведенный в таблицу приложения. На рис. 2 показан результат фильтрации записей по текстовому запросу с подсветкой найденных строк. На рис. 3 — диалог подтверждения удаления выбранной записи.

The screenshot shows a Qt application window with a search bar at the top, a table of records, and a form at the bottom. The search bar contains the text "Поиск по имени, фамилии или ID-карте..." and a button "Применить фильтр". The table has five columns: ID, Фамилия, Имя, Телефон, and ID-карта. The form has four input fields labeled "Имя:", "Фамилия:", "Телефон:", and "ID-карта:". At the bottom, there are two buttons: "Сохранить запись" and "Удалить выбранное".

	ID	Фамилия	Имя	Телефон	ID-карта
1	3	Мария	Сидорова	+79993334455	345678
2	4	Елена	Смирнова	+79994445566	456789
3	6	Ежик	Соник	+123456	ID-1001
4	7	Стив	Майнкрафт	+1234056	ID-10021
5	8	CSharp	Test	+1234567890	ID-CSHARP-001
6	13	Иван	Иванов	+79990001122	999999
7	14	Мистер	Крипер	80082	6767

Имя:

Фамилия:

Телефон:

ID-карта:

Рис. 1 – Отображение полного списка записей

ID-CSHARP-001

Применить фильтр

	ID	Фамилия	Имя	Телефон	ID-карта
1	8	CSharp	Test	+1234567890	ID-CSHARP-001

Имя:
Фамилия:
Телефон:
ID-карта:

Сохранить запись

Удалить выбранное

Рис. 2 – Поиск и подсветка найденных записей

ID-CSHARP-001

Применить фильтр

	ID	Фамилия	Имя	Телефон	ID-карта
1	8	CSharp	Test	+1234567890	ID-CSHARP-001

Имя:
Фамилия:
Телефон:
ID-карта:

Сохранить запись

Удалить выбранное

?

Вы действительно уверены?

Yes

No

Рис. 3 – Диалог подтверждения удаления записи

Вывод

В ходе выполнения лабораторной работы было разработано графическое приложение на языке C#, реализующее выборку, фильтрацию, добавление и удаление записей в базе данных PostgreSQL. Освоены механизмы вызова функций PL/pgSQL и обработка исключений при взаимодействии с СУБД.